

# SIMATS ENGINEERING

## ASSESSMENT TOOL 2

COURSE CODE: CSA14

COURSE NAME: Compiler Design

### COURSE OUTCOME COVERED

**CO4:** Examine intermediate code generation techniques to enhance code optimization (BL4)

Assessment Tool Weightage: 20%

QUESTIONS: 5

Duration :60 Mins  
On Class  
Total Marks:50

Annexure A

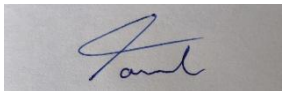
### INDUSTRY PROBLEM- SOLVING TASK

#### Code of Conduct:

*I Tamil Elakiya S ( 192524443 ) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

*I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:



| Q. No                 | Problem Understanding(20) | Method Selection(20) | Solution Process(25) | Accuracy of Calculation (20) | Interpretation of Results(15) | Total Score (out of 100) |
|-----------------------|---------------------------|----------------------|----------------------|------------------------------|-------------------------------|--------------------------|
| 1                     | / 4                       | / 4                  | / 4                  | / 4                          | / 4                           |                          |
| 2                     | / 4                       | / 4                  | / 4                  | / 4                          | / 4                           |                          |
| 3                     | / 4                       | / 4                  | / 4                  | / 4                          | / 4                           |                          |
| 4                     | / 4                       | / 4                  | / 4                  | / 4                          | / 4                           |                          |
| 5                     | / 4                       | / 4                  | / 4                  | / 4                          | / 4                           |                          |
| OVERALL RUBRICS VALUE |                           |                      |                      |                              |                               |                          |
|                       | / 4                       | / 4                  | / 4                  | / 4                          | / 4                           |                          |
|                       | / 25                      | / 25                 | / 20                 | / 20                         | / 10                          | / 100                    |

## Annexure A

### INDUSTRY PROBLEM- SOLVING TASK

#### Q1. Smart Electricity Billing System

**Concepts Covered:** Intermediate Languages, Assignment Statements, Optimization

##### Problem Statement

An electricity billing system calculates the total payable amount using units consumed, rate per unit, and tax.

$$\text{energy\_cost} = \text{units} \times \text{rate}$$
$$\text{tax\_amount} = \text{energy\_cost} \times \text{tax\_rate}$$
$$\text{total\_bill} = \text{energy\_cost} + \text{tax\_amount}$$

##### Tasks

- (i) Write suitable declarations for all variables with appropriate data types.
- (ii) Generate three-address intermediate code for the billing calculation.
- (iii) Represent the intermediate code using quadruples, triples, and indirect triples.
- (iv) Identify common subexpressions and optimization opportunities.
- (v) Produce the optimized intermediate code with justification. (10 Marks)

#### Q2. Employee Promotion Evaluation System

**Concepts Covered:** Boolean Expressions, Backpatching

##### Problem Statement

An employee is eligible for promotion if:

Experience  $\geq$  5 years

AND (Performance Score > 90 OR Certification = TRUE)

### **Tasks**

- (i) Formulate the Boolean expression.
- (ii) Generate intermediate code using short-circuit evaluation.
- (iii) Apply backpatching to resolve forward jumps.
- (iv) Generate the final labelled intermediate code.
- (v) Draw the control-flow graph for the evaluation process. (10 Marks)

### **Q3. Hospital Patient Priority System**

**Concepts Covered:** Case Statements, Intermediate Languages

#### **Problem Statement**

A hospital assigns treatment priority based on emergency level.

```
switch(priority)

{

case 1: CRITICAL;

case 2: HIGH;

case 3: MEDIUM;

case 4: LOW;

}
```

### **Tasks**

- (i) Convert the switch statement into three-address code using conditional jumps.
- (ii) Generate an equivalent jump-table implementation.
- (iii) Represent the implementation using quadruples.
- (iv) Compare conditional branching with jump-table execution.

- (v) Recommend the most suitable implementation for emergency hospitals with justification.  
(10 Marks)

#### **Q4. E-Commerce Payment Processing System**

**Concepts Covered:** Procedure Calls

##### **Problem Statement**

An online shopping application processes payments through the following procedures.

amount = calculateAmount(cart)

processPayment(amount)

sendConfirmation(amount)

##### **Tasks**

- (i) Generate intermediate code.
- (ii) Illustrate parameter passing.
- (iii) Design the activation record.
- (iv) Draw stack diagrams for call and return operations.
- (v) Explain how procedure calls are managed during execution. (10 Marks)

#### **Q5. Smart ATM Withdrawal Verification System**

**Concepts Covered:** Boolean Expressions, Backpatching

##### **Problem Statement**

A customer can withdraw cash if:

Account Balance  $\geq$  Withdrawal Amount

AND (PIN Verified = TRUE OR Biometric Verified = TRUE)

##### **Tasks**

- (i) Write the Boolean expression.

- (ii) Generate short-circuit intermediate code.
- (iii) Apply backpatching.
- (iv) Generate the final labelled intermediate code.
- (v) Draw the control-flow graph. (10 Marks)

# Smart Electricity Billing System:

Given:

$$\text{energy-cost} = \text{units} \times \text{rate}$$

$$\text{tax-amount} = \text{energy-cost} \times \text{tax-rate}$$

$$\text{total-bill} = \text{energy-cost} + \text{tax-amount}$$

i) Suitable declarations:

int units;

float rate;

float tax-rate;

float energy-cost;

float tax-amount;

float total-bill;

ii) Three-Address Code:

$$T_1 = \text{units} \times \text{rate}$$

$$T_2 = T_1 \times \text{tax-rate}$$

$$T_3 = T_1 + T_2$$

$$\text{energy-cost} = T_1$$

$$\text{tax-amount} = T_2$$

$$\text{total-bill} = T_3$$

$\therefore T_1, T_2, T_3$  are temporary variables.

iii) Quadruples:

| location | Operator | Arg 1 | Arg 2    | Result      |
|----------|----------|-------|----------|-------------|
| (0)      | *        | units | rate     | $T_1$       |
| (1)      | *        | $T_1$ | tax-rate | $T_2$       |
| (2)      | +        | $T_1$ | $T_2$    | $T_3$       |
| (3)      | =        | $T_1$ | -        | energy-cost |
| (4)      | =        | $T_2$ | -        | tax-amount  |
| (5)      | =        | $T_3$ | -        | total-bill  |

### Indirect: Triples:

|     | Location | Operator | Arg 1 | Arg 2       |
|-----|----------|----------|-------|-------------|
| 100 | (0)      | *        | units | rate        |
| 101 | (1)      | *        | (0)   | tax-rate    |
| 102 | (2)      | +        | (0)   | (1)         |
| 103 | (3)      | =        | (0)   | energy-cost |
| 104 | (4)      | =        | (1)   | tax-amount  |
| 105 | (5)      | =        | (2)   | total-bill. |

- iv) There is no repeated common subexpression in the original calculation.  $\rightarrow$  \* Common Subexpression elimination.
- \* Unnecessary temporary variables can be reduced.
  - \* Directly use calculated values where possible.
  - \* If separate assignments are not required, they can be avoided.

Optimized TAC: (Only if total-bill is alone required):

$$T_1 = \text{units} * \text{rate}$$

$$T_2 = T_1 + \text{tax-rate}$$

$$T_3 = T_1 * T_2$$

$$\text{total-bill} = T_3$$

$\therefore$  The optimized code avoids recalculating units x rate and reduces unnecessary assignments, thereby reducing the number of intermediate operations.



# Employee Promotion Evaluation System:

## Condition:

Experience  $\geq 5$  AND

(Performance Score  $> 90$  OR Certification = TRUE)

## i) Boolean expression:

Let:

E = experience  $\geq 5$

P = performance - score  $> 90$

C = certification == TRUE

$\therefore E \text{ AND } (P \text{ OR } C)$

## ii) Intermediate code:

1. if experience  $\geq 5$  goto 3
2. goto 8
3. if performance - score  $> 90$  goto 7
4. if certification == TRUE goto 7
5. goto 8
6. goto 8
7. promotion = TRUE
8. promotion = FALSE

## A cleaner labelled version is:

L1: if experience  $\geq 5$  goto L2  
goto L5

L2: if performance - score  $> 90$  goto L4  
goto L3

L3: if certification == TRUE goto L4  
goto L5

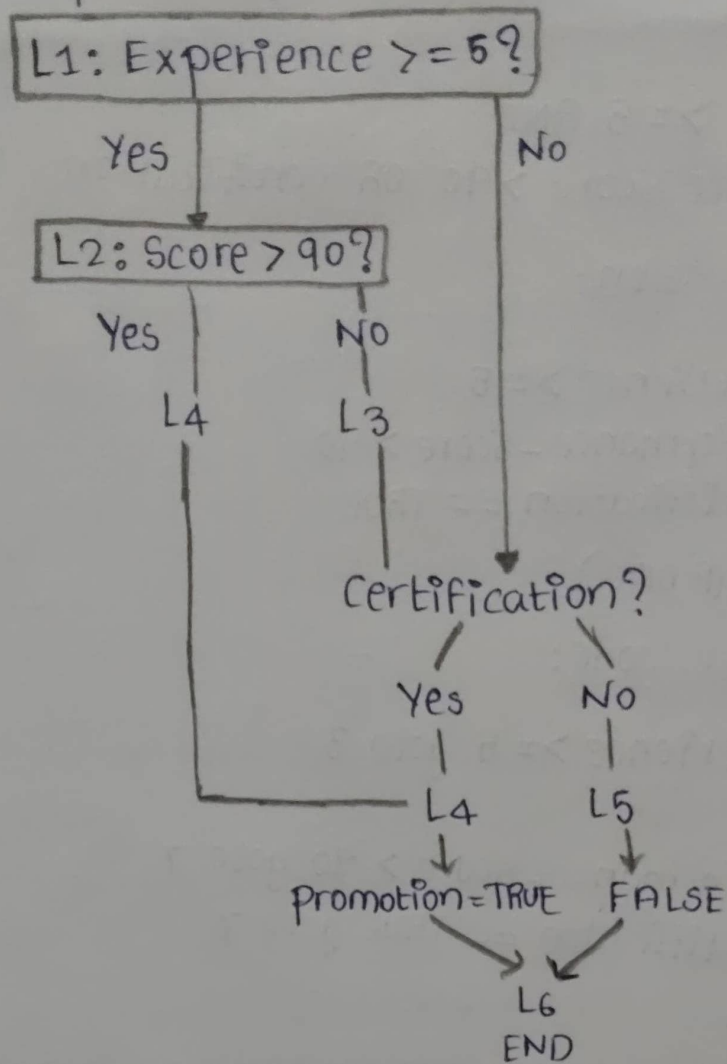
L4: promotion = TRUE  
goto L6

L5: promotion = FALSE

L6: end.



### Control - Flow Graph:



### 3. Hospital Patient Priority System:

Given:

```
Switch (Priority)
{
  case 1: CRITICAL;
  case 2: HIGH;
  case 3: MEDIUM;
  case 4: Low;
}
```

(i) TAC using Conditional Jumps

```
if priority == 1 goto L1
if priority == 2 goto L2
if priority == 3 goto L3
if priority == 4 goto L4
goto L5
```

L1: Status = CRITICAL

goto L5

L2: Status = HIGH

goto L5

L3: Status = MEDIUM

goto L5

L4: Status = LOW

L5: END

ii) Equivalent Jump-Table Implementation:

$T_1 = \text{priority} - 1$

goto JumpTable[ $T_1$ ]

Jump table:

JumpTable[0]  $\rightarrow$  L1

JumpTable[1]  $\rightarrow$  L2

JumpTable[2]  $\rightarrow$  L3

JumpTable[3]  $\rightarrow$  L4

iii) Quadruples:

| No. | Operator | Arg 1    | Arg 2 | Result |
|-----|----------|----------|-------|--------|
| 1.  | IF==     | Priority | 1     | L1     |
| 2.  | IF==     | priority | 2     | L2     |
| 3.  | IF==     | priority | 3     | L3     |
| 4.  | IF==     | priority | 4     | L4     |
| 5.  | GOTO     | -        | -     | L5     |
| 6.  | =        | CRITICAL | -     | Status |
| 7.  | GOTO     | -        | -     | L5     |
| 8.  | =        | HIGH     | -     | Status |
| 9.  | GOTO     | -        | -     | L5     |
| 10. | =        | MEDIUM   | -     | Status |
| 11. | GOTO     | -        | -     | L5     |
| 12. | =        | Low      | -     | Status |

| iv) | Feature   | Conditional Branching  | Jump Table          |
|-----|-----------|------------------------|---------------------|
|     | Method    | Multiple Comparisons   | Direct Indexed Jump |
|     | Execution | Sequential comparisons | Almost direct       |
|     | Best for  | Sparse cases           | Dense case ranges   |
|     | Memory    | Low                    | Requires table      |
|     | speed     | can be slower          | usually faster      |
|     | Example   | Cases 1, 10, 100       | Cases 1, 2, 3, 4.   |

#### 4. E-Commerce Payment Processing System:

Given:

```

amount = CalculateAmount(cart)
ProcessPayment(amount)
SendConfirmation(amount).

```

##### (i) Intermediate Code:

```

PARAM cart
CALL calculateAmount, 1
amount = RETVAL

PARAM amount
CALL processPayment, 1

PARAM amount
CALL sendConfirmation, 1.

```

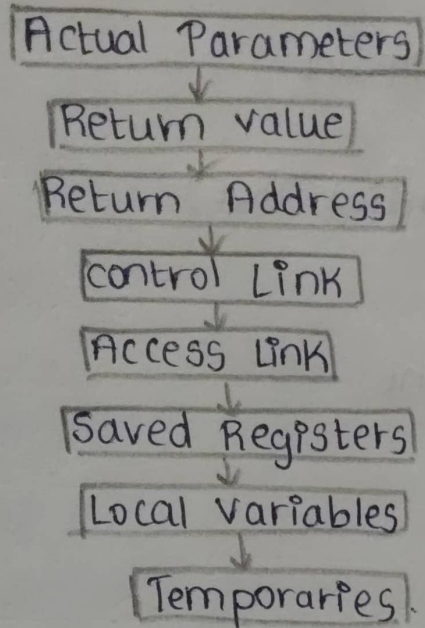
##### Three-address representation:

1. PARAM cart
2. T1 = CALL calculateAmount, 1
3. amount = T1
4. PARAM amount
5. CALL processpayment, 1
6. PARAM amount
7. CALL sendConfirmation, 1.



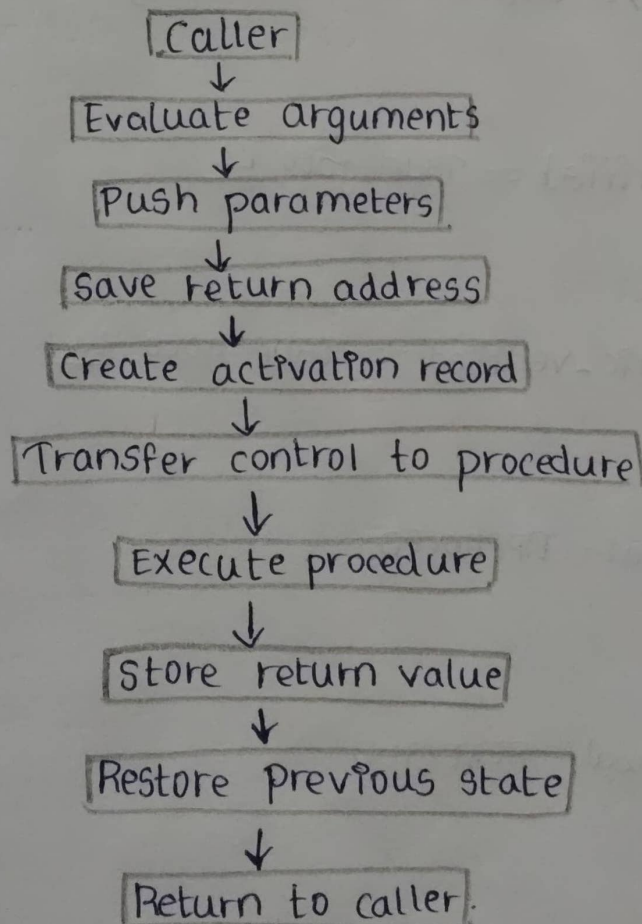
## Activation Record:

General activation record:



## iv) Procedure call Management:

Procedure calls are managed through the run-time stack.



## 5. SMART ATM Withdrawal Verification System:

Given condition:

Withdrawal is allowed if:

Account Balance  $\geq$  Withdrawal Amount

AND

(PIN verified = TRUE OR Biometric verified = TRUE)

### i) Boolean Expression:

Let:

B = balance  $\geq$  withdrawal-amount

P = pin-verified == TRUE

Bio = biometric-verified == TRUE

$\therefore$  B AND (P OR Bio)

### ii) Intermediate code:

L1:

if balance  $\geq$  withdrawal-amount goto L2  
goto L5

L2:

if pin-verified == TRUE goto L4  
goto L3

L3:

if biometric-verified == TRUE goto L4  
goto L5

L4:

withdrawal = APPROVED  
goto L6

L5:

withdrawal = REJECTED

L6:

END

(OR)

1. If balance  $\geq$  withdrawal - amount goto 3
2. goto 8
3. if pin-verified == TRUE goto 1
4. goto 5
5. if biometric-verified == TRUE goto 1
6. goto 8
7. withdrawal = APPROVED
8. withdrawal = REJECTED

Control - Flow Graph:

